

Escuela Politécnica Nacional

Nombre: Kevin Eduardo Garcia Rodriguez

Tema: [Tarea 12] Ejercicios Unidad 05-A | ODE Método de Euler

Repositorio GIT: <https://github.com/Nattyrd/Metodos-Numericos-2025B>

1. Use el método de Euler para aproximar las soluciones para cada uno de los siguientes problemas de valor inicial

a. $y' = te^{3t} - 2y$, $0 \leq t \leq 1$, $y(0) = 0$, con $h = 0.5$

b. $y' = 1 + (t - y)^2$, $2 \leq t \leq 3$, $y(2) = 1$, con $h = 0.5$

c. $y' = 1 + yt$, $1 \leq t \leq 2$, $y(1) = 2$, con $h = 0.25$

d. $y' = \cos 2t + \sin 3t$, $0 \leq t \leq 1$, $y(0) = 1$, con $h = 0.25$

```
In [5]: import numpy as np
import pandas as pd

def euler_method(f, t0, y0, h, t_end):
    """
    Resuelve una ecuación diferencial ordinaria usando el método de Euler.

    Args:
        f (function): La función f(t, y) que define la EDO, y' = f(t, y).
        t0 (float): El valor inicial de t.
        y0 (float): El valor inicial de y.
        h (float): El tamaño del paso.
        t_end (float): El valor final de t.

    Returns:
        pd.DataFrame: Un DataFrame con los valores de 't' y 'y' calculados.
    """

    # Calcular el número de pasos. Usamos round para evitar problemas de coma flotante
    num_steps = int(round((t_end - t0) / h))

    # Inicializar las listas para los resultados
    t_values = [t0]
    y_values = [y0]

    # Inicializar las variables para la iteración
    t_current = t0
    y_current = y0

    # Imprimir el encabezado de la tabla
    print(f"\n--- Método de Euler de t={t0} a t={t_end} con paso h={h} ---\n")
    print(f"{'Paso':<5} | {'t':<10} | {'y':<15}")
    print("-" * 35)
    print(f"{0:<5} | {t_current:<10.4f} | {y_current:<15.6f}")
```

```

# Bucle principal del método de Euler
for i in range(num_steps):
    # Fórmula de Euler:  $y_{n+1} = y_n + h * f(t_n, y_n)$ 
    y_next = y_current + h * f(t_current, y_current)
    t_next = t_current + h

    # Actualizar los valores y guardar en las listas
    y_current = y_next
    t_current = t_next
    t_values.append(t_current)
    y_values.append(y_current)

    # Imprimir el resultado del paso actual
    print(f"{i+1:<5} | {t_current:<10.4f} | {y_current:<15.6f}")

return pd.DataFrame({"t": t_values, "y": y_values})

```

```

In [ ]: # --- Ejecución de los casos de prueba ---

# a.  $y' = t * e^{(3t)} - 2y$ 
print("\n--- Resolviendo la Ecuación (a) ---")
f_a = lambda t, y: t * np.exp(3 * t) - 2 * y
df_a = euler_method(f_a, t0=0, y0=0, h=0.5, t_end=1)
print("\nDataFrame para la Ecuación (a):")
print(df_a)

print("\n" + "="*50 + "\n")

# b.  $y' = 1 + (t - y)^2$ 
print("\n--- Resolviendo la Ecuación (b) ---")
f_b = lambda t, y: 1 + (t - y)**2
df_b = euler_method(f_b, t0=2, y0=1, h=0.5, t_end=3)
print("\nDataFrame para la Ecuación (b):")
print(df_b)

print("\n" + "="*50 + "\n")

```

--- Resolviendo la Ecuación (a) ---

--- Método de Euler de $t=0$ a $t=1$ con paso $h=0.5$ ---

Paso	t	y

0	0.0000	0.000000
1	0.5000	0.000000
2	1.0000	1.120422

DataFrame para la Ecuación (a):

	t	y
0	0.0	0.000000
1	0.5	0.000000
2	1.0	1.120422

=====

--- Resolviendo la Ecuación (b) ---

--- Método de Euler de $t=2$ a $t=3$ con paso $h=0.5$ ---

Paso	t	y

0	2.0000	1.000000
1	2.5000	2.000000
2	3.0000	2.625000

DataFrame para la Ecuación (b):

	t	y
0	2.0	1.000
1	2.5	2.000
2	3.0	2.625

=====

--- Resolviendo la Ecuación (c) ---

--- Método de Euler de $t=1$ a $t=2$ con paso $h=0.25$ ---

Paso	t	y

0	1.0000	2.000000
1	1.2500	2.750000
2	1.5000	3.550000
3	1.7500	4.391667
4	2.0000	5.269048

DataFrame para la Ecuación (c):

	t	y
0	1.00	2.000000
1	1.25	2.750000
2	1.50	3.550000
3	1.75	4.391667
4	2.00	5.269048

=====

--- Resolviendo la Ecuación (d) ---

--- Método de Euler de $t=0$ a $t=1$ con paso $h=0.25$ ---

Paso	t	y
0	0.0000	1.000000
1	0.2500	1.250000
2	0.5000	1.639805
3	0.7500	2.024255
4	1.0000	2.236457

DataFrame para la Ecuación (d):

	t	y
0	0.00	1.000000
1	0.25	1.250000
2	0.50	1.639805
3	0.75	2.024255
4	1.00	2.236457

```
In [7]: # c.  $y' = 1 + y / t$ 
print("\n--- Resolviendo la Ecuación (c) ---")
f_c = lambda t, y: 1 + y / t
df_c = euler_method(f_c, t0=1, y0=2, h=0.25, t_end=2)
print("\nDataFrame para la Ecuación (c):")
print(df_c)

print("\n" + "="*50 + "\n")

# d.  $y' = \cos(2t) + \sin(3t)$ 
print("\n--- Resolviendo la Ecuación (d) ---")
f_d = lambda t, y: np.cos(2 * t) + np.sin(3 * t)
df_d = euler_method(f_d, t0=0, y0=1, h=0.25, t_end=1)
print("\nDataFrame para la Ecuación (d):")
print(df_d)
```

--- Resolviendo la Ecuación (c) ---

--- Método de Euler de t=1 a t=2 con paso h=0.25 ---

Paso	t	y
0	1.0000	2.000000
1	1.2500	2.750000
2	1.5000	3.550000
3	1.7500	4.391667
4	2.0000	5.269048

DataFrame para la Ecuación (c):

	t	y
0	1.00	2.000000
1	1.25	2.750000
2	1.50	3.550000
3	1.75	4.391667
4	2.00	5.269048

=====

--- Resolviendo la Ecuación (d) ---

--- Método de Euler de t=0 a t=1 con paso h=0.25 ---

Paso	t	y
0	0.0000	1.000000
1	0.2500	1.250000
2	0.5000	1.639805
3	0.7500	2.024255
4	1.0000	2.236457

DataFrame para la Ecuación (d):

	t	y
0	0.00	1.000000
1	0.25	1.250000
2	0.50	1.639805
3	0.75	2.024255
4	1.00	2.236457

2. Las soluciones reales para los problemas de valor inicial en el ejercicio 1 se proporcionan aquí. Compare el error real en cada paso.

a. $y(t) = \frac{1}{5}te^{3t} - \frac{1}{25}e^{3t} + \frac{1}{25}e^{-2t}$
 c. $y(t) = t \ln t + 2t$

b. $y(t) = t + \frac{1}{1-t}$
 d. $y(t) = \frac{1}{2}\sin 2t - \frac{1}{3}\cos 3t + \frac{4}{3}$

```
In [8]: import numpy as np
import pandas as pd

# Método de Euler
def euler_method(f, t0, y0, h, t_end):
    """
    Resuelve una ecuación diferencial ordinaria usando el método de Euler.
```

```

Args:
    f (function): La función  $f(t, y)$  que define la EDO,  $y' = f(t, y)$ .
    t0 (float): El valor inicial de  $t$ .
    y0 (float): El valor inicial de  $y$ .
    h (float): El tamaño del paso.
    t_end (float): El valor final de  $t$ .

Returns:
    pd.DataFrame: Un DataFrame con los valores de ' $t$ ' y ' $y$ ' calculados.
"""
steps = int((t_end - t0) / h) + 1
t_values = [t0]
y_values = [y0]
t = t0
y = y0
for _ in range(steps - 1):
    y += h * f(t, y)
    t += h
    t_values.append(t)
    y_values.append(y)
return pd.DataFrame({"t": t_values, "y": y_values})

# Funciones de derivadas (problema 1)
f_a = lambda t, y: t * np.exp(3 * t) - 2 * y
f_b = lambda t, y: 1 + (t - y)**2
f_c = lambda t, y: 1 + y / t
f_d = lambda t, y: np.cos(2 * t) + np.sin(3 * t)

# Aplicar método de Euler a cada problema
df_a = euler_method(f_a, 0, 0, 0.5, 1)
df_b = euler_method(f_b, 2, 1, 0.5, 3)
df_c = euler_method(f_c, 1, 2, 0.25, 2)
df_d = euler_method(f_d, 0, 1, 0.25, 1)

# Funciones exactas (problema 2)
def exact_a(t):
    return (1/5)*t*np.exp(3*t) - (1/25)*np.exp(3*t) + (1/25)*np.exp(-2*t)

def exact_b(t):
    return t + 1/(1 - t)

def exact_c(t):
    return t * np.log(t) + 2 * t

def exact_d(t):
    return 0.5 * np.sin(2*t) - (1/3) * np.cos(3*t) + 4/3

# Comparar con solución exacta
def comparar(df, exact_func):
    df["y_exacta"] = exact_func(df["t"])
    df["error"] = np.abs(df["y"] - df["y_exacta"])
    return df[["t", "y", "y_exacta", "error"]]

# Calcular errores
df_a_err = comparar(df_a, exact_a)
df_b_err = comparar(df_b, exact_b)
df_c_err = comparar(df_c, exact_c)
df_d_err = comparar(df_d, exact_d)

# Mostrar resultados

```

```
print("\n--- a. Error Método de Euler ---")
print(df_a_err)
print("\n--- b. Error Método de Euler ---")
print(df_b_err)
print("\n--- c. Error Método de Euler ---")
print(df_c_err)
print("\n--- d. Error Método de Euler ---")
print(df_d_err)
```

--- a. Error Método de Euler ---

	t	y	y_exacta	error
0	0.0	0.000000	0.000000	0.000000
1	0.5	0.000000	0.283617	0.283617
2	1.0	1.120422	3.219099	2.098677

--- b. Error Método de Euler ---

	t	y	y_exacta	error
0	2.0	1.000	1.000000	0.000000
1	2.5	2.000	1.833333	0.166667
2	3.0	2.625	2.500000	0.125000

--- c. Error Método de Euler ---

	t	y	y_exacta	error
0	1.00	2.000000	2.000000	0.000000
1	1.25	2.750000	2.778929	0.028929
2	1.50	3.550000	3.608198	0.058198
3	1.75	4.391667	4.479328	0.087661
4	2.00	5.269048	5.386294	0.117247

--- d. Error Método de Euler ---

	t	y	y_exacta	error
0	0.00	1.000000	1.000000	0.000000
1	0.25	1.250000	1.329150	0.079150
2	0.50	1.639805	1.730490	0.090684
3	0.75	2.024255	2.041472	0.017217
4	1.00	2.236457	2.117980	0.118478

3. Utilice el método de Euler para aproximar las soluciones para cada uno de los siguientes problemas de valor inicial

- $y' = y/t - (y/t)^4, 1 \leq t \leq 2, y(1) = 1, \text{con } h = 0.1$
- $y' = 1 + y/t + (y/t)^2, 1 \leq t \leq 3, y(1) = 0, \text{con } h = 0.2$
- $y' = -(y+1)(y+3), 0 \leq t \leq 2, y(0) = -2, \text{con } h = 0.2$
- $y' = -5y + 5t^2 + 2t, 0 \leq t \leq 1, y(0) = \frac{1}{3}, \text{con } h = 0.1$

In [9]:

```
import numpy as np
import pandas as pd

# Método de Euler
def euler_method(f, t0, y0, h, t_end):
    """
    Resuelve una ecuación diferencial ordinaria usando el método de Euler.

    Args:
        f (function): La función f(t, y) que define la EDO, y' = f(t, y).
        t0 (float): El valor inicial de t.
        y0 (float): El valor inicial de y.
        h (float): El tamaño del paso.
```

```

    t_end (float): El valor final de t.

Returns:
    pd.DataFrame: Un DataFrame con los valores de 't' y 'y' calculados.
"""
steps = int((t_end - t0) / h) + 1
t_values = [t0]
y_values = [y0]
t = t0
y = y0
for _ in range(steps - 1):
    y += h * f(t, y)
    t += h
    t_values.append(t)
    y_values.append(y)
return pd.DataFrame({"t": t_values, "y": y_values})

# Definimos las funciones de derivada de cada inciso

# a.  $y' = y/t - (y/t)^2$ ,  $t$  in  $[1, 2]$ ,  $y(1) = 1$ ,  $h = 0.1$ 
f_a = lambda t, y: (y / t) - (y / t) ** 2
df_a = euler_method(f_a, t0=1, y0=1, h=0.1, t_end=2)

# b.  $y' = 1 + y/t + (y/t)^2$ ,  $t$  in  $[1, 3]$ ,  $y(1) = 0$ ,  $h = 0.2$ 
f_b = lambda t, y: 1 + y / t + (y / t) ** 2
df_b = euler_method(f_b, t0=1, y0=0, h=0.2, t_end=3)

# c.  $y' = -(y + 1)(y + 3)$ ,  $t$  in  $[0, 2]$ ,  $y(0) = -2$ ,  $h = 0.2$ 
f_c = lambda t, y: - (y + 1) * (y + 3)
df_c = euler_method(f_c, t0=0, y0=-2, h=0.2, t_end=2)

# d.  $y' = -5y + 5t^2 + 2t$ ,  $t$  in  $[0, 1]$ ,  $y(0) = 1/3$ ,  $h = 0.1$ 
f_d = lambda t, y: -5 * y + 5 * t**2 + 2 * t
df_d = euler_method(f_d, t0=0, y0=1/3, h=0.1, t_end=1)

# Mostrar los resultados
print("\n--- a. Resultado Método de Euler ---")
print(df_a)
print("\n--- b. Resultado Método de Euler ---")
print(df_b)
print("\n--- c. Resultado Método de Euler ---")
print(df_c)
print("\n--- d. Resultado Método de Euler ---")
print(df_d)

```


--- a. Resultado Método de Euler ---

	t	y
0	1.0	1.000000
1	1.1	1.000000
2	1.2	1.008264
3	1.3	1.021689
4	1.4	1.038515
5	1.5	1.057668
6	1.6	1.078461
7	1.7	1.100432
8	1.8	1.123262
9	1.9	1.146724
10	2.0	1.170652

--- b. Resultado Método de Euler ---

	t	y
0	1.0	0.000000
1	1.2	0.200000
2	1.4	0.438889
3	1.6	0.721243
4	1.8	1.052038
5	2.0	1.437251
6	2.2	1.884261
7	2.4	2.402270
8	2.6	3.002837
9	2.8	3.700601
10	3.0	4.514277

--- c. Resultado Método de Euler ---

	t	y
0	0.0	-2.000000
1	0.2	-1.800000
2	0.4	-1.608000
3	0.6	-1.438733
4	0.8	-1.301737
5	1.0	-1.199251
6	1.2	-1.127491
7	1.4	-1.079745
8	1.6	-1.049119
9	1.8	-1.029954
10	2.0	-1.018152

--- d. Resultado Método de Euler ---

	t	y
0	0.0	0.333333
1	0.1	0.166667
2	0.2	0.108333
3	0.3	0.114167
4	0.4	0.162083
5	0.5	0.241042
6	0.6	0.345521
7	0.7	0.472760
8	0.8	0.621380
9	0.9	0.790690
10	1.0	0.980345

4. Aquí se dan las soluciones reales para los problemas de valor inicial en el ejercicio 3.
Calcule el error real en las aproximaciones del ejercicio 3.

$$\begin{aligned} \text{a. } y(t) &= \frac{t}{1+\ln t} \\ \text{c. } y(t) &= -3 + \frac{2}{1+e^{-2t}} \end{aligned}$$

$$\begin{aligned} \text{b. } y(t) &= t \tan(\ln t) \\ \text{d. } y(t) &= t^2 + \frac{1}{3}e^{-5t} \end{aligned}$$

```
In [ ]: import numpy as np
import pandas as pd

# Método de Euler
def euler_method(f, t0, y0, h, t_end):
    """
    Resuelve una ecuación diferencial ordinaria usando el método de Euler.

    Args:
        f (function): La función f(t, y) que define la EDO, y' = f(t, y).
        t0 (float): El valor inicial de t.
        y0 (float): El valor inicial de y.
        h (float): El tamaño del paso.
        t_end (float): El valor final de t.

    Returns:
        pd.DataFrame: Un DataFrame con los valores de 't' y 'y' calculados.
    """
    steps = int((t_end - t0) / h) + 1
    t_values = [t0]
    y_values = [y0]
    t = t0
    y = y0
    for _ in range(steps - 1):
        y += h * f(t, y)
        t += h
        t_values.append(t)
        y_values.append(y)
    return pd.DataFrame({"t": t_values, "y_aprox": y_values})

# Aproximaciones del ejercicio 3

# a. y' = y/t - (y/t)^2, t in [1, 2], y(1) = 1, h = 0.1
f_a = lambda t, y: (y / t) - (y / t) ** 2
df_a = euler_method(f_a, t0=1, y0=1, h=0.1, t_end=2)

# b. y' = 1 + y/t + (y/t)^2, t in [1, 3], y(1) = 0, h = 0.2
f_b = lambda t, y: 1 + y / t + (y / t) ** 2
df_b = euler_method(f_b, t0=1, y0=0, h=0.2, t_end=3)

# c. y' = -(y + 1)(y + 3), t in [0, 2], y(0) = -2, h = 0.2
f_c = lambda t, y: - (y + 1) * (y + 3)
df_c = euler_method(f_c, t0=0, y0=-2, h=0.2, t_end=2)

# d. y' = -5y + 5t^2 + 2t, t in [0, 1], y(0) = 1/3, h = 0.1
f_d = lambda t, y: -5 * y + 5 * t**2 + 2 * t
df_d = euler_method(f_d, t0=0, y0=1/3, h=0.1, t_end=1)

# Soluciones exactas del ejercicio 4
def exact_a(t):
    return t / (1 + np.log(t))
def exact_b(t):
    return t * np.tan(np.log(t))
def exact_c(t):
    return -3 + 2 / (1 + np.exp(-2 * t))
def exact_d(t):
    return t**2 + 1/3 * np.exp(-5*t)
```

```
    return t**2 + (1/3) * np.exp(-5 * t)

# Comparación función
def comparar(df, exact_func):
    df["y_exacta"] = exact_func(df["t"])
    df["error"] = np.abs(df["y_aprox"] - df["y_exacta"])
    return df
```

```
In [11]: # Comparar cada resultado
df_a_res = comparar(df_a, exact_a)
df_b_res = comparar(df_b, exact_b)
df_c_res = comparar(df_c, exact_c)
df_d_res = comparar(df_d, exact_d)

# Mostrar resultados
pd.set_option("display.precision", 6)
print("\n--- a. Comparación ---")
print(df_a_res)
print("\n--- b. Comparación ---")
print(df_b_res)
print("\n--- c. Comparación ---")
print(df_c_res)
print("\n--- d. Comparación ---")
print(df_d_res)
```

--- a. Comparación ---

	t	y_aprox	y_exacta	error
0	1.0	1.000000	1.000000	0.000000
1	1.1	1.000000	1.004282	0.004282
2	1.2	1.008264	1.014952	0.006688
3	1.3	1.021689	1.029814	0.008124
4	1.4	1.038515	1.047534	0.009019
5	1.5	1.057668	1.067262	0.009594
6	1.6	1.078461	1.088433	0.009972
7	1.7	1.100432	1.110655	0.010223
8	1.8	1.123262	1.133654	0.010392
9	1.9	1.146724	1.157228	0.010505
10	2.0	1.170652	1.181232	0.010581

--- b. Comparación ---

	t	y_aprox	y_exacta	error
0	1.0	0.000000	0.000000	0.000000
1	1.2	0.200000	0.221243	0.021243
2	1.4	0.438889	0.489682	0.050793
3	1.6	0.721243	0.812753	0.091510
4	1.8	1.052038	1.199439	0.147401
5	2.0	1.437251	1.661282	0.224031
6	2.2	1.884261	2.213502	0.329241
7	2.4	2.402270	2.876551	0.474282
8	2.6	3.002837	3.678475	0.675638
9	2.8	3.700601	4.658665	0.958064
10	3.0	4.514277	5.874100	1.359823

--- c. Comparación ---

	t	y_aprox	y_exacta	error
0	0.0	-2.000000	-2.000000	0.000000
1	0.2	-1.800000	-1.802625	0.002625
2	0.4	-1.608000	-1.620051	0.012051
3	0.6	-1.438733	-1.462950	0.024218
4	0.8	-1.301737	-1.335963	0.034226
5	1.0	-1.199251	-1.238406	0.039155
6	1.2	-1.127491	-1.166345	0.038854
7	1.4	-1.079745	-1.114648	0.034903
8	1.6	-1.049119	-1.078331	0.029212
9	1.8	-1.029954	-1.053194	0.023240
10	2.0	-1.018152	-1.035972	0.017821

--- d. Comparación ---

	t	y_aprox	y_exacta	error
0	0.0	0.333333	0.333333	0.000000
1	0.1	0.166667	0.212177	0.045510
2	0.2	0.108333	0.162626	0.054293
3	0.3	0.114167	0.164377	0.050210
4	0.4	0.162083	0.205112	0.043028
5	0.5	0.241042	0.277362	0.036320
6	0.6	0.345521	0.376596	0.031075
7	0.7	0.472760	0.500066	0.027305
8	0.8	0.621380	0.646105	0.024725
9	0.9	0.790690	0.813703	0.023013
10	1.0	0.980345	1.002246	0.021901

5. Utilice los resultados del ejercicio 3 y la interpolación lineal para aproximar los siguientes valores de $y(t)$. Compare las aproximaciones asignadas para los valores reales obtenidos mediante las funciones determinadas en el ejercicio 4.

- a. $y(0.25)$ y $y(0.93)$
- b. $y(t) = y(1.25)$ y $y(1.93)$
- c. $y(2.10)$ y $y(2.75)$
- d. $y(t) = y(0.54)$ y $y(0.94)$

```
In [12]: import numpy as np
import pandas as pd

# Método de Euler
def euler_method(f, t0, y0, h, t_end):
    """
    Resuelve una ecuación diferencial ordinaria usando el método de Euler.

    Args:
        f (function): La función f(t, y) que define la EDO, y' = f(t, y).
        t0 (float): El valor inicial de t.
        y0 (float): El valor inicial de y.
        h (float): El tamaño del paso.
        t_end (float): El valor final de t.

    Returns:
        pd.DataFrame: Un DataFrame con los valores de 't' y 'y' calculados.
    """
    steps = int((t_end - t0) / h) + 1
    t_values = [t0]
    y_values = [y0]
    t = t0
    y = y0
    for _ in range(steps - 1):
        y += h * f(t, y)
        t += h
        t_values.append(t)
        y_values.append(y)
    return pd.DataFrame({"t": t_values, "y_aprox": y_values})

# Funciones del ejercicio 3
f_a = lambda t, y: (y / t) - (y / t)**2
f_b = lambda t, y: 1 + (y / t) + (y / t)**2
f_c = lambda t, y: -(y + 1)*(y + 3)
f_d = lambda t, y: -5*y + 5*t**2 + 2*t

# Aproximaciones con Euler
df_a = euler_method(f_a, t0=1, y0=1, h=0.1, t_end=2)
df_b = euler_method(f_b, t0=1, y0=0, h=0.2, t_end=3)
df_c = euler_method(f_c, t0=0, y0=-2, h=0.2, t_end=2)
df_d = euler_method(f_d, t0=0, y0=1/3, h=0.1, t_end=1)

# Interpolación lineal
def interpolar(df, t_eval):
    return np.interp(t_eval, df["t"], df["y_aprox"])

# Soluciones exactas del ejercicio 4
def exact_a(t): return t / (1 + np.log(t))
def exact_b(t): return t * np.tan(np.log(t))
def exact_c(t): return -3 + 2 / (1 + np.exp(-2 * t))
```

```
def exact_d(t): return t**2 + (1/3) * np.exp(-5 * t)

# Puntos a interpolar
puntos = {
    'a': [0.25, 0.93],
    'b': [1.25, 1.93],
    'c': [2.10, 2.75],
    'd': [0.54, 0.94]
}

# Asociación de incisos
casos = {
    'a': (df_c, exact_c), # [0,2]
    'b': (df_a, exact_a), # [1,2]
    'c': (df_b, exact_b), # [1,3]
    'd': (df_d, exact_d), # [0,1]
}

# Mostrar resultados
for letra in ['a', 'b', 'c', 'd']:
    df, f_exacta = casos[letra]
    print(f"\n--- Inciso {letra.upper()} ---")
    for t_eval in puntos[letra]:
        y_interp = interpolar(df, t_eval)
        y_real = f_exacta(t_eval)
        error = abs(y_interp - y_real)
        print(f"t = {t_eval:.2f} | y_interp = {y_interp:.6f} | y_real = {y_real:.6f} | error = {error:.6f}")
```

--- Inciso A ---

t = 0.25 | y_interp = -1.752000 | y_real = -1.755081 | error = 0.003081

t = 0.93 | y_interp = -1.235121 | y_real = -1.269406 | error = 0.034285

--- Inciso B ---

t = 1.25 | y_interp = 1.014977 | y_real = 1.021957 | error = 0.006980

t = 1.93 | y_interp = 1.153902 | y_real = 1.164390 | error = 0.010488

--- Inciso C ---

t = 2.10 | y_interp = 1.660756 | y_real = 1.924962 | error = 0.264206

t = 2.75 | y_interp = 3.526160 | y_real = 4.394170 | error = 0.868010

--- Inciso D ---

t = 0.54 | y_interp = 0.282833 | y_real = 0.314002 | error = 0.031169

t = 0.94 | y_interp = 0.866552 | y_real = 0.886632 | error = 0.020080

6. Use el método de Taylor de orden 2 para aproximar las soluciones para cada uno de los siguientes problemas de valor inicial.

a. $y' = te^{3t} - 2y$, $0 \leq t \leq 1$, $y(0) = 0$, con $h = 0.5$

b. $y' = 1 + (t - y)^2$, $2 \leq t \leq 3$, $y(2) = 1$, con $h = 0.5$

c. $y' = 1 + yt$, $1 \leq t \leq 2$, $y(1) = 2$, con $h = 0.25$

d. $y' = \cos 2t + \sin 3t$, $0 \leq t \leq 1$, $y(0) = 1$, con $h = 0.25$

In [14]: `import numpy as np`
`import pandas as pd`

```

# Método de Euler
def euler_method(f, t0, y0, h, t_end):
    """
    Resuelve una ecuación diferencial ordinaria usando el método de Euler.

    Args:
        f (function): La función f(t, y) que define la EDO,  $y' = f(t, y)$ .
        t0 (float): El valor inicial de t.
        y0 (float): El valor inicial de y.
        h (float): El tamaño del paso.
        t_end (float): El valor final de t.

    Returns:
        pd.DataFrame: Un DataFrame con los valores de 't' y 'y' calculados.
    """
    steps = int((t_end - t0) / h) + 1
    t_values = [t0]
    y_values = [y0]
    t = t0
    y = y0
    for _ in range(steps - 1):
        y += h * f(t, y)
        t += h
        t_values.append(t)
        y_values.append(y)
    return pd.DataFrame({"t": t_values, "y_aprox": y_values})

# Funciones del ejercicio 3
f_a = lambda t, y: (y / t) - (y / t)**2
f_b = lambda t, y: 1 + (y / t) + (y / t)**2
f_c = lambda t, y: -(y + 1)*(y + 3)
f_d = lambda t, y: -5*y + 5*t**2 + 2*t

# Aproximaciones con Euler
df_a_euler = euler_method(f_a, t0=1, y0=1, h=0.1, t_end=2)
df_b_euler = euler_method(f_b, t0=1, y0=0, h=0.2, t_end=3)
df_c_euler = euler_method(f_c, t0=0, y0=-2, h=0.2, t_end=2)
df_d_euler = euler_method(f_d, t0=0, y0=1/3, h=0.1, t_end=1)

# Interpolación lineal
def interpolar(df, t_eval):
    return np.interp(t_eval, df["t"], df["y_aprox"])

# Soluciones exactas del ejercicio 4
def exact_a(t): return t / (1 + np.log(t))
def exact_b(t): return t * np.tan(np.log(t))
def exact_c(t): return -3 + 2 / (1 + np.exp(-2 * t))
def exact_d(t): return t**2 + (1/3) * np.exp(-5 * t)

# Puntos a interpolar
puntos = {
    'a': [0.25, 0.93],
    'b': [1.25, 1.93],
    'c': [2.10, 2.75],
    'd': [0.54, 0.94]
}

# Asociación de incisos
casos = {
    'a': (df_c_euler, exact_c), # [0,2]

```

```

    'b': (df_a_euler, exact_a), # [1,2]
    'c': (df_b_euler, exact_b), # [1,3]
    'd': (df_d_euler, exact_d), # [0,1]
}

# Mostrar resultados de Euler
print("\n--- Resultados Método de Euler ---")
for letra in ['a', 'b', 'c', 'd']:
    df, f_exacta = casos[letra]
    print(f"\nInciso {letra.upper()}:")
    for t_eval in puntos[letra]:
        y_interp = interpolar(df, t_eval)
        y_real = f_exacta(t_eval)
        error = abs(y_interp - y_real)
        print(f"t = {t_eval:.2f} | y_interp = {y_interp:.6f} | y_real = {y_real:.6f} | error = {error:.6f}")

# Método de Taylor de orden 2
def taylor_orden_2(f, df_dt, df_dy, t0, y0, h, t_end):
    t_values = [t0]
    y_values = [y0]
    t = t0
    y = y0
    steps = int((t_end - t0) / h)
    for _ in range(steps):
        f_val = f(t, y)
        df_val = df_dt(t, y) + df_dy(t, y) * f_val
        y = y + h * f_val + (h**2 / 2) * df_val
        t = t + h
        t_values.append(t)
        y_values.append(y)
    return pd.DataFrame({"t": t_values, "y_aprox": y_values})

# a.  $y' = t * e^{(3t)} - 2y$ 
f_a_taylor = lambda t, y: t * np.exp(3 * t) - 2 * y
dfdt_a = lambda t, y: (1 + 3 * t) * np.exp(3 * t)
dfdy_a = lambda t, y: -2
df_a_taylor = taylor_orden_2(f_a_taylor, dfdt_a, dfdy_a, t0=0, y0=0, h=0.5, t_end=1)

# b.  $y' = 1 + (t - y)^2$ 
f_b_taylor = lambda t, y: 1 + (t - y)**2
dfdt_b = lambda t, y: 2 * (t - y)
dfdy_b = lambda t, y: -2 * (t - y)
df_b_taylor = taylor_orden_2(f_b_taylor, dfdt_b, dfdy_b, t0=2, y0=1, h=0.5, t_end=3)

# c.  $y' = 1 + y / t$ 
f_c_taylor = lambda t, y: 1 + y / t
dfdt_c = lambda t, y: -y / (t**2)
dfdy_c = lambda t, y: 1 / t
df_c_taylor = taylor_orden_2(f_c_taylor, dfdt_c, dfdy_c, t0=1, y0=2, h=0.25, t_end=2)

# d.  $y' = \cos(2t) + \sin(3t)$ 
f_d_taylor = lambda t, y: np.cos(2 * t) + np.sin(3 * t)
dfdt_d = lambda t, y: -2 * np.sin(2 * t) + 3 * np.cos(3 * t)
dfdy_d = lambda t, y: 0
df_d_taylor = taylor_orden_2(f_d_taylor, dfdt_d, dfdy_d, t0=0, y0=1, h=0.25, t_end=1)

# Mostrar resultados
print("\n--- Resultados Método de Taylor Orden 2 ---")
for letra, df in zip("abcd", [df_a_taylor, df_b_taylor, df_c_taylor, df_d_taylor]):
    print(f"\nInciso {letra}:")
    for t_eval in puntos[letra]:
        y_interp = interpolar(df, t_eval)
        y_real = f_exacta(t_eval)
        error = abs(y_interp - y_real)
        print(f"t = {t_eval:.2f} | y_interp = {y_interp:.6f} | y_real = {y_real:.6f} | error = {error:.6f}")

```



```
print(f"\nInciso {letra.upper()}:")
print(df)
```

--- Resultados Método de Euler ---

Inciso A:

t = 0.25		y_interp = -1.752000		y_real = -1.755081		error = 0.003081
t = 0.93		y_interp = -1.235121		y_real = -1.269406		error = 0.034285

Inciso B:

t = 1.25		y_interp = 1.014977		y_real = 1.021957		error = 0.006980
t = 1.93		y_interp = 1.153902		y_real = 1.164390		error = 0.010488

Inciso C:

t = 2.10		y_interp = 1.660756		y_real = 1.924962		error = 0.264206
t = 2.75		y_interp = 3.526160		y_real = 4.394170		error = 0.868010

Inciso D:

t = 0.54		y_interp = 0.282833		y_real = 0.314002		error = 0.031169
t = 0.94		y_interp = 0.866552		y_real = 0.886632		error = 0.020080

--- Resultados Método de Taylor Orden 2 ---

Inciso A:

	t	y_aprox
0	0.0	0.000000
1	0.5	0.125000
2	1.0	2.023239

Inciso B:

	t	y_aprox
0	2.0	1.000000
1	2.5	1.750000
2	3.0	2.425781

Inciso C:

	t	y_aprox
0	1.00	2.000000
1	1.25	2.781250
2	1.50	3.612500
3	1.75	4.485417
4	2.00	5.394048

Inciso D:

	t	y_aprox
0	0.00	1.000000
1	0.25	1.343750
2	0.50	1.772187
3	0.75	2.110676
4	1.00	2.201644

7. Repita el ejercicio 6 con el método de Taylor de orden 4

```
In [15]: import numpy as np
import pandas as pd

# Método de Euler
def euler_method(f, t0, y0, h, t_end):
    """
```

Resuelve una ecuación diferencial ordinaria usando el método de Euler.

Args:

`f` (function): La función $f(t, y)$ que define la EDO, $y' = f(t, y)$.
`t0` (float): El valor inicial de t .
`y0` (float): El valor inicial de y .
`h` (float): El tamaño del paso.
`t_end` (float): El valor final de t .

Returns:

`pd.DataFrame`: Un DataFrame con los valores de ' t ' y ' y ' calculados.

```
"""
steps = int((t_end - t0) / h) + 1
t_values = [t0]
y_values = [y0]
t = t0
y = y0
for _ in range(steps - 1):
    y += h * f(t, y)
    t += h
    t_values.append(t)
    y_values.append(y)
return pd.DataFrame({"t": t_values, "y_aprox": y_values})

# Funciones del ejercicio 3
f_a = lambda t, y: (y / t) - (y / t)**2
f_b = lambda t, y: 1 + (y / t) + (y / t)**2
f_c = lambda t, y: -(y + 1)*(y + 3)
f_d = lambda t, y: -5*y + 5*t**2 + 2*t

# Aproximaciones con Euler
df_a_euler = euler_method(f_a, t0=1, y0=1, h=0.1, t_end=2)
df_b_euler = euler_method(f_b, t0=1, y0=0, h=0.2, t_end=3)
df_c_euler = euler_method(f_c, t0=0, y0=-2, h=0.2, t_end=2)
df_d_euler = euler_method(f_d, t0=0, y0=1/3, h=0.1, t_end=1)

# Interpolación lineal
def interpolar(df, t_eval):
    return np.interp(t_eval, df["t"], df["y_aprox"])

# Soluciones exactas del ejercicio 4
def exact_a(t): return t / (1 + np.log(t))
def exact_b(t): return t * np.tan(np.log(t))
def exact_c(t): return -3 + 2 / (1 + np.exp(-2 * t))
def exact_d(t): return t**2 + (1/3) * np.exp(-5 * t)

# Puntos a interpolar
puntos = {
    'a': [0.25, 0.93],
    'b': [1.25, 1.93],
    'c': [2.10, 2.75],
    'd': [0.54, 0.94]
}

# Asociación de incisos
casos = {
    'a': (df_c_euler, exact_c), # [0,2]
    'b': (df_a_euler, exact_a), # [1,2]
    'c': (df_b_euler, exact_b), # [1,3]
    'd': (df_d_euler, exact_d), # [0,1]
```

```

}

# Mostrar resultados de Euler
print("\n--- Resultados Método de Euler ---")
for letra in ['a', 'b', 'c', 'd']:
    df, f_exacta = casos[letra]
    print(f"\nInciso {letra.upper()}:")
    for t_eval in puntos[letra]:
        y_interp = interpolar(df, t_eval)
        y_real = f_exacta(t_eval)
        error = abs(y_interp - y_real)
        print(f"t = {t_eval:.2f} | y_interp = {y_interp:.6f} | y_real = {y_real:.6f}")

# Método de Taylor de orden 2
def taylor_orden_2(f, df_dt, df_dy, t0, y0, h, t_end):
    t_values = [t0]
    y_values = [y0]
    t = t0
    y = y0
    steps = int((t_end - t0) / h)
    for _ in range(steps):
        f_val = f(t, y)
        df_val = df_dt(t, y) + df_dy(t, y) * f_val
        y = y + h * f_val + (h**2 / 2) * df_val
        t = t + h
        t_values.append(t)
        y_values.append(y)
    return pd.DataFrame({"t": t_values, "y_aprox": y_values})

# a.  $y' = t * e^{(3t)} - 2y$ 
f_a_taylor_2 = lambda t, y: t * np.exp(3 * t) - 2 * y
dfdt_a = lambda t, y: (1 + 3 * t) * np.exp(3 * t)
dfdy_a = lambda t, y: -2
df_a_taylor_2 = taylor_orden_2(f_a_taylor_2, dfdt_a, dfdy_a, t0=0, y0=0, h=0.5,

# b.  $y' = 1 + (t - y)^2$ 
f_b_taylor_2 = lambda t, y: 1 + (t - y)**2
dfdt_b = lambda t, y: 2 * (t - y)
dfdy_b = lambda t, y: -2 * (t - y)
df_b_taylor_2 = taylor_orden_2(f_b_taylor_2, dfdt_b, dfdy_b, t0=2, y0=1, h=0.5,

# c.  $y' = 1 + y / t$ 
f_c_taylor_2 = lambda t, y: 1 + y / t
dfdt_c = lambda t, y: -y / (t**2)
dfdy_c = lambda t, y: 1 / t
df_c_taylor_2 = taylor_orden_2(f_c_taylor_2, dfdt_c, dfdy_c, t0=1, y0=2, h=0.25,

# d.  $y' = \cos(2t) + \sin(3t)$ 
f_d_taylor_2 = lambda t, y: np.cos(2 * t) + np.sin(3 * t)
dfdt_d = lambda t, y: -2 * np.sin(2 * t) + 3 * np.cos(3 * t)
dfdy_d = lambda t, y: 0
df_d_taylor_2 = taylor_orden_2(f_d_taylor_2, dfdt_d, dfdy_d, t0=0, y0=1, h=0.25,

# Mostrar resultados
print("\n--- Resultados Método de Taylor Orden 2 ---")
for letra, df in zip("abcd", [df_a_taylor_2, df_b_taylor_2, df_c_taylor_2, df_d_taylor_2]):
    print(f"\nInciso {letra.upper()}:")
    print(df)

# Método de Taylor de orden 4

```

```

def taylor_orden_4(f, f1, f2, f3, t0, y0, h, t_end):
    t_values = [t0]
    y_values = [y0]
    t, y = t0, y0
    steps = int((t_end - t0) / h)

    for _ in range(steps):
        f_val = f(t, y)
        f1_val = f1(t, y)
        f2_val = f2(t, y)
        f3_val = f3(t, y)

        y = y + h * f_val + (h**2 / 2) * f1_val + (h**3 / 6) * f2_val + (h**4 / 24) * f3_val
        t += h
        t_values.append(t)
        y_values.append(y)

    return pd.DataFrame({"t": t_values, "y_aprox": y_values})

# ===== a.  $y' = t \cdot e^{(3t)} - 2y$  =====

f_a_taylor_4 = lambda t, y: t*np.exp(3*t) - 2*y
f1_a = lambda t, y: (1 + 3*t)*np.exp(3*t) - 2*f_a_taylor_4(t, y)
f2_a = lambda t, y: 6*np.exp(3*t) + 9*(1 + 3*t)*np.exp(3*t) - 2*f1_a(t, y)
f3_a = lambda t, y: (27*(1 + 3*t) + 54)*np.exp(3*t) - 2*f2_a(t, y)

df_a_taylor_4 = taylor_orden_4(f_a_taylor_4, f1_a, f2_a, f3_a, t0=0, y0=0, h=0.5)

# ===== b.  $y' = 1 + (t - y)^2$  =====

f_b_taylor_4 = lambda t, y: 1 + (t - y)**2
f1_b = lambda t, y: 2*(t - y)*(1 - f_b_taylor_4(t, y))
f2_b = lambda t, y: 2*(1 - f_b_taylor_4(t, y))**2 - 2*(t - y)*f1_b(t, y)
f3_b = lambda t, y: (-4*(1 - f_b_taylor_4(t, y))*f1_b(t, y) - 2*f1_b(t, y)*(1 - f_b_taylor_4(t, y)))

df_b_taylor_4 = taylor_orden_4(f_b_taylor_4, f1_b, f2_b, f3_b, t0=2, y0=1, h=0.5)

# ===== c.  $y' = 1 + y/t$  =====

f_c_taylor_4 = lambda t, y: 1 + y / t
f1_c = lambda t, y: -y / t**2 + (1 / t) * f_c_taylor_4(t, y)
f2_c = lambda t, y: (2 * y) / t**3 - 2 * f_c_taylor_4(t, y) / t**2 + (1 / t) * f1_c(t, y)
f3_c = lambda t, y: (-6 * y) / t**4 + 6 * f_c_taylor_4(t, y) / t**3 - 3 * f1_c(t, y)

df_c_taylor_4 = taylor_orden_4(f_c_taylor_4, f1_c, f2_c, f3_c, t0=1, y0=2, h=0.2)

# ===== d.  $y' = \cos(2t) + \sin(3t)$  =====

f_d_taylor_4 = lambda t, y: np.cos(2*t) + np.sin(3*t)
f1_d = lambda t, y: -2*np.sin(2*t) + 3*np.cos(3*t)
f2_d = lambda t, y: -4*np.cos(2*t) - 9*np.sin(3*t)
f3_d = lambda t, y: 8*np.sin(2*t) - 27*np.cos(3*t)

df_d_taylor_4 = taylor_orden_4(f_d_taylor_4, f1_d, f2_d, f3_d, t0=0, y0=1, h=0.2)

# ===== Resultados =====

print("\n--- Resultados Método de Taylor de orden 4 ---")
for letra, df in zip("abcd", [df_a_taylor_4, df_b_taylor_4, df_c_taylor_4, df_d_taylor_4]):
    print(f"\nInciso {letra.upper()}: \n{df}")

```

--- Resultados Método de Euler ---

Inciso A:

t = 0.25 | y_interp = -1.752000 | y_real = -1.755081 | error = 0.003081
 t = 0.93 | y_interp = -1.235121 | y_real = -1.269406 | error = 0.034285

Inciso B:

t = 1.25 | y_interp = 1.014977 | y_real = 1.021957 | error = 0.006980
 t = 1.93 | y_interp = 1.153902 | y_real = 1.164390 | error = 0.010488

Inciso C:

t = 2.10 | y_interp = 1.660756 | y_real = 1.924962 | error = 0.264206
 t = 2.75 | y_interp = 3.526160 | y_real = 4.394170 | error = 0.868010

Inciso D:

t = 0.54 | y_interp = 0.282833 | y_real = 0.314002 | error = 0.031169
 t = 0.94 | y_interp = 0.866552 | y_real = 0.886632 | error = 0.020080

--- Resultados Método de Taylor Orden 2 ---

Inciso A:

	t	y_aprox
0	0.0	0.000000
1	0.5	0.125000
2	1.0	2.023239

Inciso B:

	t	y_aprox
0	2.0	1.000000
1	2.5	1.750000
2	3.0	2.425781

Inciso C:

	t	y_aprox
0	1.00	2.000000
1	1.25	2.781250
2	1.50	3.612500
3	1.75	4.485417
4	2.00	5.394048

Inciso D:

	t	y_aprox
0	0.00	1.000000
1	0.25	1.343750
2	0.50	1.772187
3	0.75	2.110676
4	1.00	2.201644

--- Resultados Método de Taylor de orden 4 ---

Inciso A:

	t	y_aprox
0	0.0	0.000000
1	0.5	0.539062
2	1.0	5.366595

Inciso B:

	t	y_aprox
0	2.0	1.000000
1	2.5	1.875000

2 3.0 2.528351

Inciso C:

	t	y_aprox
--	---	---------

0	1.00	2.000000
---	------	----------

1	1.25	2.778971
---	------	----------

2	1.50	3.608266
---	------	----------

3	1.75	4.479416
---	------	----------

4	2.00	5.386400
---	------	----------

Inciso D:

	t	y_aprox
--	---	---------

0	0.00	1.000000
---	------	----------

1	0.25	1.328939
---	------	----------

2	0.50	1.729667
---	------	----------

3	0.75	2.039934
---	------	----------

4	1.00	2.115988
---	------	----------